

Preentrenamiento

De la web cruda al modelo base: datos y curación, scaling laws, y qué es —y qué no— un modelo base.

IA Generativa Avanzada y Sistemas Multi-Agente

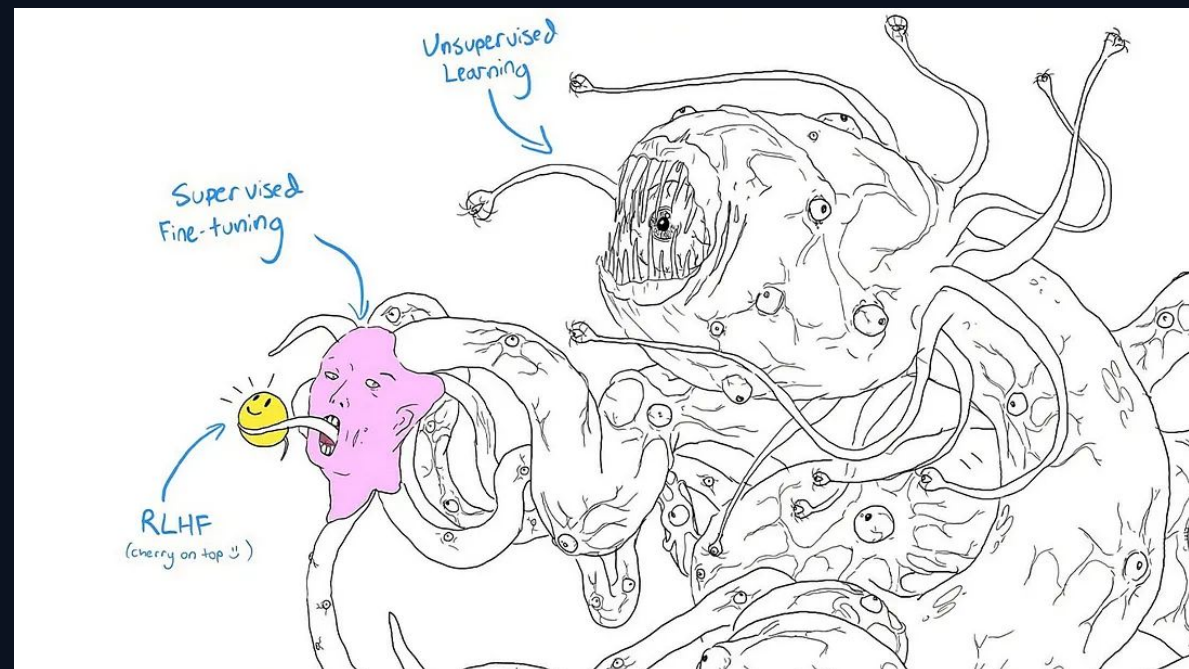
Licenciatura en Tecnología Digital · Universidad Torcuato Di Tella



De la masa al asistente

Lo que ves. El unsupervised learning (preentrenamiento) construye una masa gigantesca de patrones — tentáculos y ojos por todos lados, sin forma reconocible. El supervised fine-tuning le da una silueta. El RLHF es la carita sonriente encima: la parte que efectivamente “hablás” cuando usás un asistente.

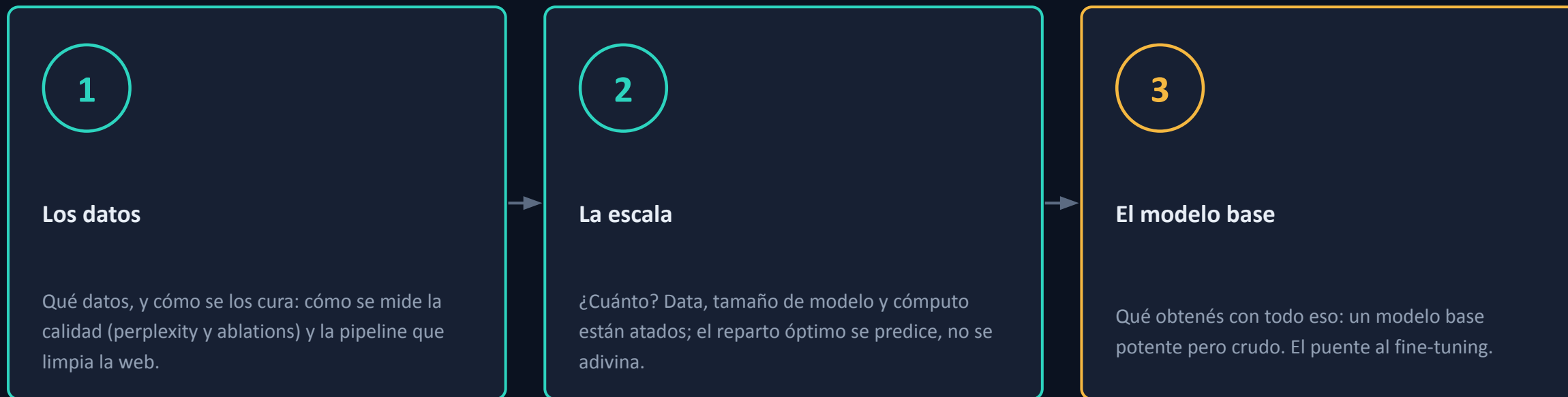
La proporción importa. Casi todo el cómputo y casi todos los datos se gastan en construir la masa. El ajuste que la vuelve “amigable” es, en comparación, una capa fina — el meme lo llama “cherry on top” con razón.



meme conocido de la comunidad de IA — pretraining, SFT y RLHF, de izquierda a derecha

El recorrido: datos, escala y modelo base

Preentrenar es ingeniería empírica: cada decisión se gana midiendo. El preentrenamiento va en tres tramos.



El hilo. los tres tramos comparten método — medir, no dogmatizar. Datos (ablations) y escala (scaling laws) son las dos mitades de la misma receta.

PREENTRENAMIENTO

Los datos

De la web cruda a texto limpio: qué datos usar, y cómo se los cura.



El algoritmo ya lo tenemos; ahora manda la data

Recap. Ya vimos cómo entrenar un Transformer. Ese mecanismo —predecir el próximo token, calcular la loss, ajustar los pesos— no cambia.

El salto. Lo que separa un GPT que balbucea de un modelo potente no es el algoritmo: es con qué datos y a qué escala se entrena.

La consecuencia. A escala, el modelo vale lo que valen sus datos. La calidad de la data compite de igual a igual con la arquitectura.



El cambio de foco. mismo algoritmo, distinto insumo. Todo este bloque es sobre el insumo: los datos y cómo se los prepara.

Cada compañía tiene su propio libro de recetas

El secreto de la industria. Los datasets de los modelos top (Llama, OpenAi, Claude) no se publican. Cada compañía tiene su propio 'libro de recetas' de curación, y lo mantiene cerrado.

Pero el esquema es común. A grandes rasgos, todas siguen los mismos pasos. Vamos a estudiar ese proceso genérico, no la fórmula de nadie en particular.

Una ventana abierta. Nos apoyamos en un proyecto open-source que sí documenta y justifica cada decisión: el ejemplo que nos deja ver un proceso normalmente oculto.

Modelos top de la industria

receta de datos = caja negra



nos deja entender

Referencia abierta

cada paso documentado y validado con experimentos

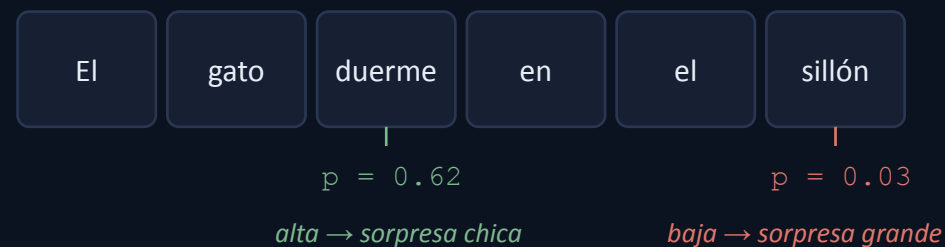
Referencia usada: FineWeb / FineWeb-2 (ver cierre).

Cómo leerlo. estudiamos cómo se piensa la curación —el proceso—, no la receta puntual de un producto.

Perplexity: de la loss a un número que se entiende

La pregunta. Un LM autoregresivo, dado un contexto, devuelve una distribución de probabilidad sobre el próximo token. Sobre un texto de test que realmente ocurrió: un buen modelo le asignó probabilidad alta a lo que efectivamente vino; uno malo se 'sorprende'.

La idea. Perplexity cuantifica esa sorpresa promedio, hecha número.



CÓMO SE LEE

El modelo avanza token a token; en cada paso miramos qué probabilidad le puso al token que de verdad vino. Promediar esa 'sorpresa' es la perplexity.

El punto de partida. no es una métrica nueva ni misteriosa: nace de la misma distribución con la que ya entrenaron el modelo.

De multiplicar probabilidades a la cross-entropy

El intento naive. $P(w_1, \dots, w_N) = \prod P(w_i \mid \text{contexto})$. La idea sería: 'más alto, mejor'.

Dos problemas. Es un producto de números menores a 1: para un texto de largo real hace underflow a cero. Y depende del largo del texto: no se pueden comparar corpus de distinto tamaño.

El arreglo. Pasar a log y promediar por token: $(1/N) \sum \log P(w_i \mid \text{contexto})$. Ya es independiente del largo — es el average log-likelihood por token.

$$\prod P(w_i \mid \text{contexto}) \rightarrow 0$$

underflow · depende de N



$$(1/N) \sum \log P(w_i \mid \text{contexto})$$

independiente del largo

La bisagra. ese promedio de log-probabilidades, con signo negativo, es exactamente la cross-entropy loss con la que ya entrenaron el modelo. No es una métrica nueva — es la misma loss, léida distinto.

Exponenciar: el branching factor efectivo

La definición. $PPL = \exp\left(-\frac{1}{N} \sum \log P(w_i | \text{contexto})\right) = \exp(\text{cross-entropy})$.

Por qué exponenciar. Le da una interpretación concreta: perplexity es el número efectivo de opciones equiprobables entre las que el modelo elige, en promedio, en cada paso.

Casos límite

Dado justo (6 caras) **PPL = 6**
tan confundido como elegir entre 6

Modelo perfecto **PPL = 1**
nunca se sorprende

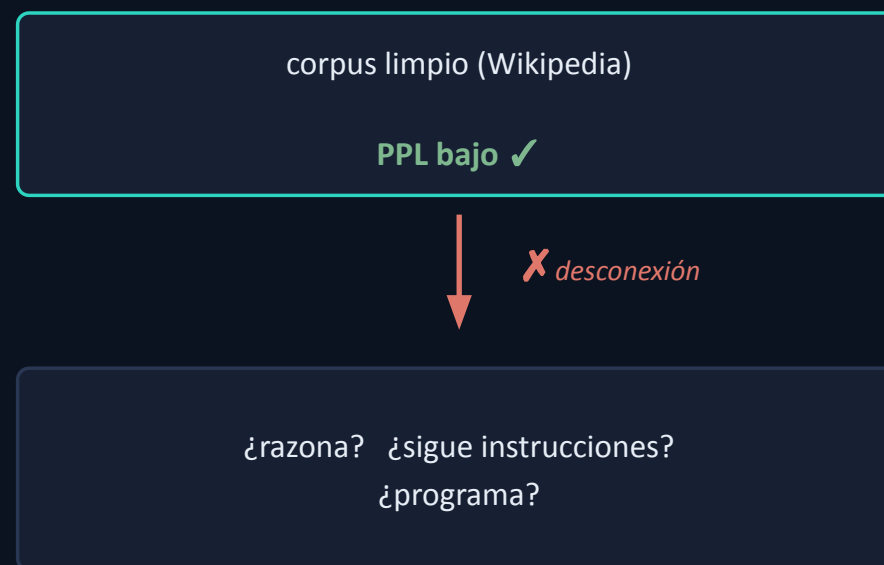
Sin entrenar (vocab V) **PPL = V**
p. ej. 50.000 → inútil

La escala. PPL va de 1 (perfecto) al tamaño del vocabulario (inútil). PPL 20 = 'elige entre 20 opciones en promedio'; más bajo, más seguro y más acertado.

Perplexity es mal juez de la data

El truco histórico. Medir la perplexity del modelo sobre un corpus de referencia 'limpio' (clásicamente Wikipedia o WikiText-103). PPL más baja = 'mejor modelo / mejor data'. Suena razonable.

Tres motivos por los que falla. (1) No compara entre tokenizers: es por token, otra tokenización da otro número. (2) Premia data que se parece al corpus de referencia → sesga hacia un estilo angosto. (3) No trackea lo que importa: PPL excelente en Wikipedia no implica razonar, seguir instrucciones ni programar.



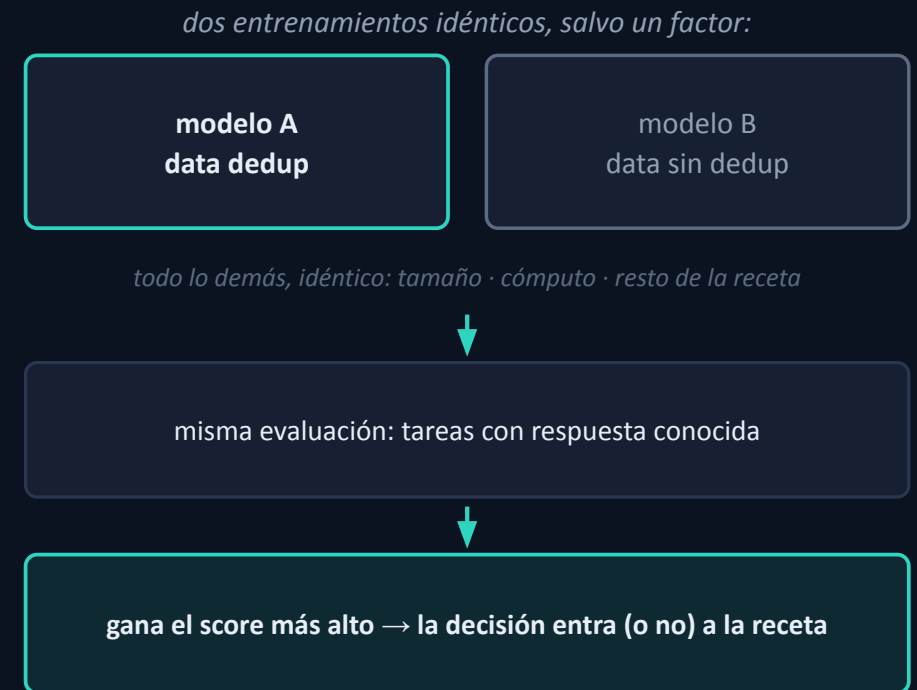
La falla. perplexity mide fluidez con un estilo de texto, no si el modelo puede hacer lo que en verdad importa. Hace falta otro juez.

Ablations: probar un cambio con un experimento

Qué es. Una ablation es un experimento controlado: cambiás una sola cosa —un filtro, una fuente, una mezcla de datos— y dejás todo lo demás fijo. Así aislás el efecto de esa decisión sola.

El ejemplo. ¿Deduplicar mejora el modelo? Entrenás dos modelos chicos idénticos —mismo tamaño, mismo cómputo, todo igual— salvo la data: uno con el corpus deduplicado, el otro sin.

Cómo se decide. Evaluás a los dos en una batería de tareas con respuesta conocida (sentido común, trivia). Si el deduplicado puntúa más alto, dedup entra a la receta; si no, se descarta.



La regla. cada decisión de curación se gana con un experimento, no con intuición: un factor cambia, todo lo demás igual.

Perplexity vs ablations

Recordá. Una ablation compara dos entrenamientos idénticos que difieren en un solo factor. Acá la usamos para juzgar la data: es el buen juez que a perplexity le falta.

En qué se separa de perplexity. En qué le pedís al modelo entrenado: perplexity le da texto corrido (la misma tarea del entrenamiento); ablations le da tareas con respuesta conocida (una tarea distinta) y mide accuracy.

El truco de costo. El modelo por dataset es chico y barato —no el modelo final—, confiando en que el ranking relativo se mantiene a escala. Caro pero manejable, no 'reentrenar el modelo grande por cada filtro'.

Perplexity	Ablations
le da al modelo	
texto corrido, nunca visto	tareas con respuesta conocida
qué mide	
next-token (misma tarea)	accuracy (tarea distinta)
qué captura	
fluidez / estilo	competencia real

Por qué ganan. perplexity pregunta '¿predice bien prosa?'; ablations, '¿resuelve lo que necesito?'. Este es el método con el que se construye toda la pipeline que sigue.

El proceso: un embudo que limpia la web

De toda la web cruda a texto limpio y de calidad. Ocho pasos; cada uno descarta un tipo distinto de basura.



El mapa. no importa el detalle de cada caja: curar es una secuencia de descartes, cada uno con su propósito. Los recorreremos agrupados por idea.

El embudo, caja por caja

1. URL

Antes de bajar la página, se descarta el dominio entero si está en una lista negra (porno, spam, malware, sitios de baja calidad conocidos). Es el filtro más barato: evita procesar basura obvia.

2. Extracción de texto

La página baja como HTML lleno de menús, botones, publicidad y código. Este paso extrae solo el texto del artículo y descarta todo el andamiaje que lo rodea.

3. Idioma

Un clasificador detecta el idioma de cada página. Según la mezcla de idiomas que se busque, la página se conserva, se descarta, o se rutea a su idioma para procesarla aparte.

4. Gopher

Reglas de calidad heurísticas (de la receta Gopher): descarta textos demasiado cortos, con exceso de símbolos o números, o con demasiada repetición interna.

5. MinHash

Detecta documentos casi idénticos —la misma nota copiada en mil sitios— y deja un solo representante, para que el modelo no vea diez veces lo mismo.

6. Filtros C4

Reglas probadas heredadas del corpus C4: saca líneas sin puntuación, placeholders tipo 'lorem ipsum', y boilerplate (avisos de cookies, pies repetidos).

7. Filtros propios

Reglas ad-hoc que cada equipo agrega para la basura específica que ve en su corpus y que los filtros genéricos no atrapan.

8. PII

Reemplaza datos personales (emails, teléfonos, direcciones IP) por placeholders, para que el modelo no memorice ni escupa información privada de alguien.

Los filtros rápidos: dirección y datos personales

Dos pasos deciden por una regla simple. Sin mucha vuelta: uno en la puerta de entrada, otro en la salida. (En el diagrama están en los extremos.)

URL — juzgar por la dirección, antes de leer. Una lista negra de dominios (porno, piratería, phishing, malware): todo lo que venga de ahí se descarta sin abrirlo. Ejemplo: el dominio entero de un sitio pirata afuera, aunque alguna página suelta tuviera texto útil.

PII — tapar datos personales, al final. Los mails y las direcciones IP se reemplazan por un placeholder. Para que el modelo no memorice y escupa el contacto de una persona real.

URL — por dominio, antes de leer

lista negra → sitiopirata.com ✗

PII — tapar datos personales

juan.perez@gmail.com → [EMAIL]

192.168.0.1 → [IP]

Descarte grueso. reglas mecánicas, sin vuelta: quién entra por la puerta y qué se tapa a la salida.

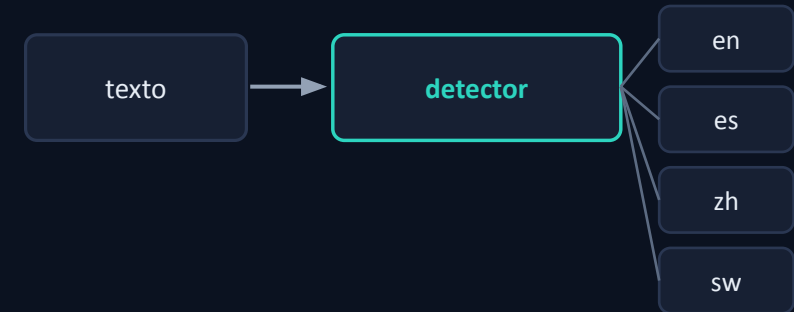
Idioma: elegir la mezcla, no quedarse con inglés

En un dataset de inglés. Un detector lee el texto y adivina el idioma con un nivel de confianza; se queda solo con inglés, y solo si está bastante seguro. Una página en español se va; una casi toda números o precios, sin prosa real, también.

En un modelo multilinguaje. No se descartan los otros idiomas: el mismo detector se usa para clasificar y rutear cada texto por idioma (y por alfabeto).

Los filtros se calibran por idioma. 'Calidad' se ve distinto en cada lengua, y hasta la noción de 'palabra' cambia (chino, japonés o tailandés no usan espacios). Un umbral global no sirve.

Y aparece la mezcla. Cuánto de cada idioma entra. Hay mucho menos texto en swahili que en inglés, así que las lenguas de pocos recursos se upsamlean (se repiten) para que el modelo no termine 95% inglés.



La mezcla (upsampling de lenguas con poca data):



Ref. multilinguaje: FineWeb-2 (1000+ idiomas).

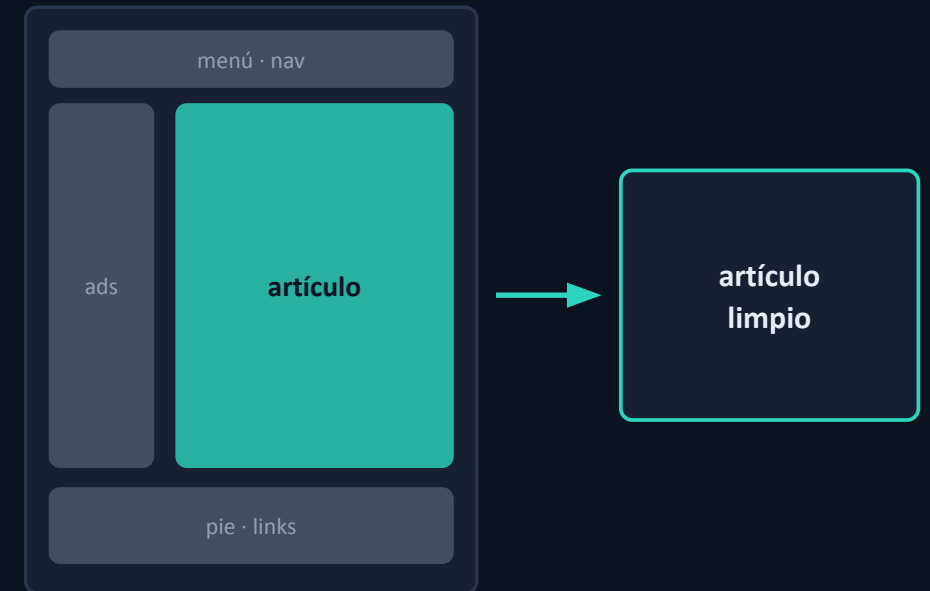
La diferencia clave. en inglés el filtro descarta; en multilinguaje clasifica y decide una mezcla, subiendo la proporción de las lenguas con poca data. La receta de Claude es cerrada; la forma es esta.

Extracción de texto: el artículo, no el decorado

La página es, en su mayoría, andamiaje. El menú de arriba (Inicio, Deportes, Contacto), el cartel de cookies, los banners de publicidad, las 'notas relacionadas', el pie con links.

Este paso se queda con el artículo. Lo que un humano vendría a leer, y tira todo el decorado.

La lección. Más texto no es mejor. Dejar el menú y los ads hace que el modelo aprenda peor.



Garbage in, garbage out. el modelo no ve la web tal cual: ve el texto que le dejamos.

Filtros de calidad: Gopher, C4 y propios

Chequeos de sentido común. Heredados de recetas como Gopher y C4. Algunos ejemplos concretos:

Palabras comunes. Un texto real en inglés está lleno de 'the', 'and', 'of'. Una lista de keywords o de precios casi no las tiene → afuera.

Repetición y símbolos. 'oferta oferta oferta' quinientas veces, o una página 90% viñetas o hashtags → afuera.

Relleno y sobras de código. Páginas con 'lorem ipsum' (texto de relleno de un diseño sin terminar) o con 'activá JavaScript para continuar' → afuera.

muestras que se descartan:

```
precio · precio · precio · $9.99 ...
```

✗

```
#deal #sale #oferta #promo #hot ...
```

✗

```
lorem ipsum dolor sit amet ...
```

✗

```
Activá JavaScript para continuar
```

✗

Curar es empírico. ninguna regla entra por linda: cada una se validó con una ablation, y a veces una regla tosca le gana a una sofisticada.

Deduplicación (MinHash): la web es una fotocopiadora

La web se repite. La misma nota de agencia en cien diarios, el artículo de Wikipedia espejado en cincuenta sitios, los 'términos y condiciones' idénticos por todos lados.

Este paso deja una sola copia. Detecta las casi-copias y colapsa cada grupo a un representante → el modelo ve más variedad con el mismo esfuerzo.

La vuelta de tuerca. Pasado cierto punto, deduplicar de más empieza a tirar lo bueno y dejar lo raro. La intuición 'más es mejor' hubo que medirla.



LA SORPRESA

Deduplicar de más no mejora: puede tirar lo bueno y conservar lo raro/fuera de distribución. Cuánto deduplicar se decide midiendo.

Testear, no creer. menos repetición ayuda... hasta que deja de ayudar. Solo el experimento lo dice.

Síntesis: curar es ciencia empírica

El embudo suma. Cada paso, justificado con un experimento, mejora un poco el modelo. Filtrar mejor rinde más que agregar más data cruda.

Lo que se llevan. Calidad > cantidad, y cada decisión se gana midiendo, no por dogma.

El puente. Ya tenemos la data. Ahora: ¿cuánta, qué tan grande el modelo, cuánto cómputo? → scaling laws.



Lo que te llevás. curar datos es ciencia empírica, no una lista de reglas heredadas.

Para profundizar

Las dos ventanas abiertas al proceso de curación que la industria mantiene cerrado. Documentan y validan cada decisión con ablations.

FineWeb — Decanting the Web

HuggingFace · el caso abierto de curación de datos web en inglés. Recorre toda la pipeline con ejemplos y ablations (15B tokens).

huggingface.co/spaces/HuggingFaceFW/blogpost-fineweb-v1

FineWeb-2 — Multilingüe

La misma pipeline adaptada a 1000+ idiomas: ruteo por idioma, filtros calibrados por lengua y la mezcla (upsampling de lenguas con poca data).

huggingface.co/datasets/HuggingFaceFW/fineweb-2

La idea que se llevan. cada compañía tiene su libro de recetas cerrado, pero el proceso es más o menos este; estas referencias abiertas nos dejaron entenderlo, y su lección de fondo es que curar datos se decide con experimentos.

PREENTRENAMIENTO

La escala

¿Cuánto? Cuánta data, qué tan grande el modelo, cuánto cómputo — y cómo se predice.



La otra mitad de la receta: ¿cuánto?

De los datos a la escala. La curación respondió qué datos usar. Pero calidad no es cantidad: falta decidir cuánto de todo — qué tan grande el modelo, cuántos datos, cuánto cómputo.

Por qué no es 'probá y fijate'. Entrenar un modelo grande cuesta del orden de millones de dólares y semanas de cómputo. Y es irreversible: si salió mal, no lo deshacés. No podés iterar a ciegas.

Lo que hace falta. Poder predecir el resultado antes de gastar. Eso es lo que dan las scaling laws.



¿cuánto de cada una?

entrenar: ~US\$ millones · irreversible

El problema. a esta escala no se prueba y se ve: se predice y después se gasta.

Las tres perillas están atadas: $C \approx 6ND$

No son independientes. El cómputo total de un entrenamiento es, con muy buena aproximación, $C \approx 6 \cdot N \cdot D$: número de parámetros por número de tokens (por una constante).

La consecuencia. Fijado el presupuesto de cómputo C , N y D se compensan: más parámetros obligan a menos tokens, y al revés. No se pueden optimizar las tres a la vez.

El reparto es el problema. Elegir cómo partir un C dado entre modelo (N) y datos (D) es la pregunta central.

$$C \approx 6 \cdot N \cdot D$$

N = parámetros del modelo (su tamaño)

D = tokens del corpus de entrenamiento

C = cómputo total del entrenamiento (FLOPs)

a C fijo, N y D se compensan:

$N \uparrow$
params



$D \downarrow$
tokens

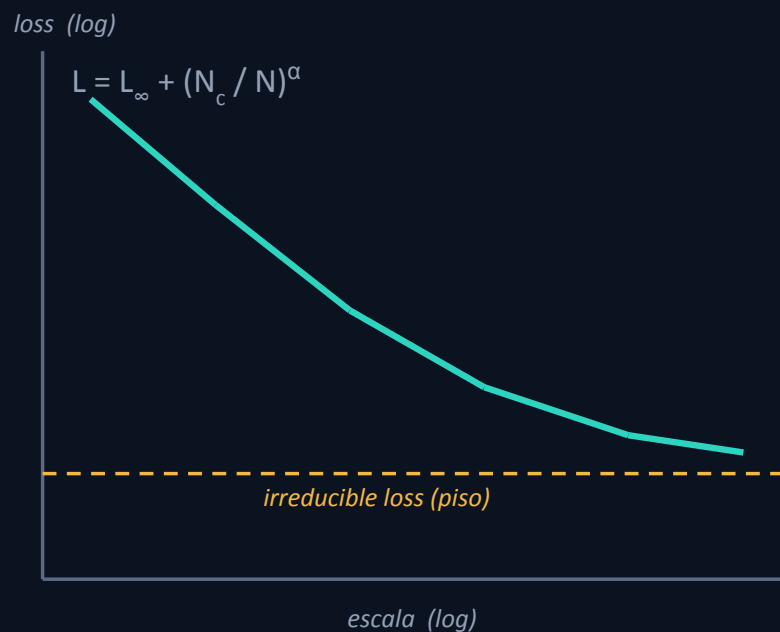
La atadura. con C fijo, subir N baja D . Toda la discusión que sigue es sobre ese trade-off.

El hallazgo: la loss baja de forma predecible

El descubrimiento empírico. Al escalar N, D o C, la loss del modelo baja siguiendo una power law: una recta en escala log-log, sostenida sobre muchos órdenes de magnitud.

Por qué es enorme. El rendimiento es suave y regular, no caótico. Escalar no da saltos impredecibles: da una curva que se puede leer.

El piso. La recta no baja para siempre: tiende a una irreducible loss, la entropía intrínseca del lenguaje que ningún modelo puede bajar.



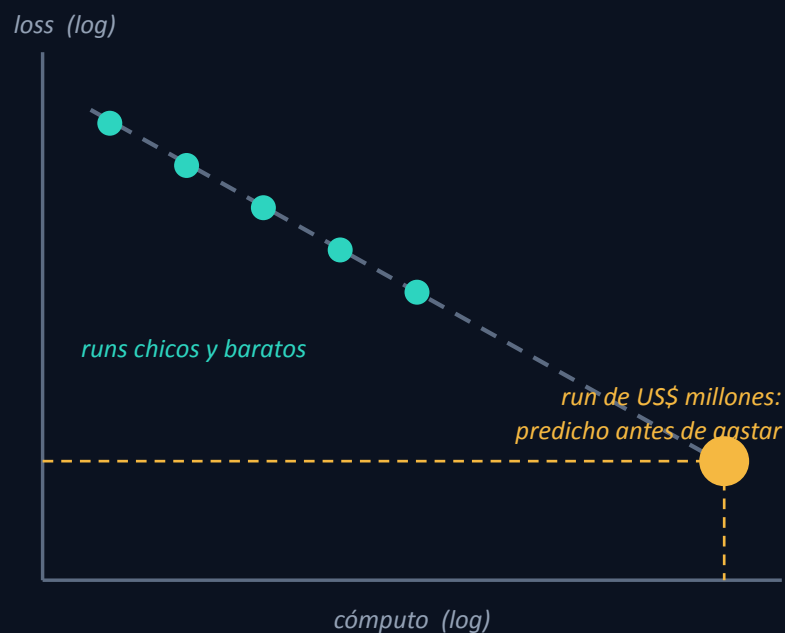
La forma. en log-log es una recta: más escala → menos loss, de manera predecible, hasta un piso.

El pago: predecir el run gigante antes de gastar

El mecanismo. Como es una power law suave, alcanza con entrenar una serie de modelos chicos y baratos, fitear la curva, y extrapolar la loss del modelo gigante.

Para qué sirve. De-riskea el run de millones: sabés a qué loss vas a llegar antes de lanzarlo. Elegís tamaño e hiperparámetros con evidencia, no con fe.

El caso real. GPT-4 reportó haber predicho su loss final desde runs hasta 1000× más chicos. La extrapolación acertó.



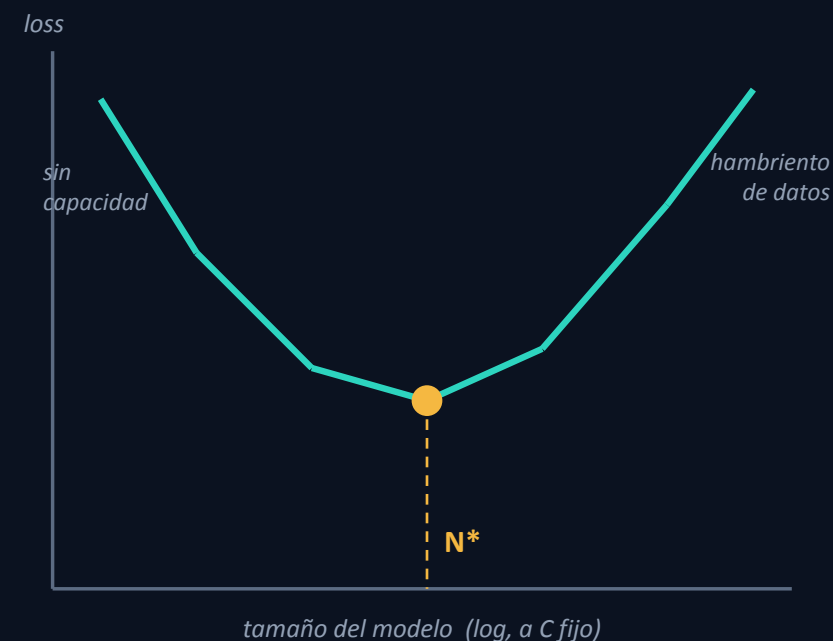
El takeaway central. fitear con lo barato y extrapolar a lo caro convierte un salto de fe en una predicción.

¿Modelo grande o mucha data? Hay un óptimo

El trade-off, con C fijo. Dado un presupuesto de cómputo, ¿lo gastás en un modelo más grande o en más tokens? Las dos cosas no: $C \approx 6ND$ las ata.

Los dos extremos fallan. Modelo muy grande con pocos datos → hambriento de datos, desperdicia capacidad. Modelo muy chico con muchos datos → no tiene capacidad para absorberlos. Se pierde por los dos lados.

En el medio, un mínimo. Para cada C hay un tamaño de modelo óptimo N^* (y su D^* correspondiente) que minimiza la loss.



El óptimo. a compute fijo, ni el más grande ni el más chico: hay un punto justo entre modelo y datos.

Chinchilla vs Gopher: la cuenta con números

El experimento que lo fijó. DeepMind (2022) entrenó dos modelos con el mismo cómputo.
Gopher: 280B parámetros, 300B tokens. Chinchilla: 70B parámetros, 1.4T tokens.

El resultado. Chinchilla, 4× más chico, le gana a Gopher en casi todo. Gopher estaba sobredimensionado y necesitado datos: gastó demasiado en parámetros y muy poco en tokens.

La regla que dejó. Para entrenamiento compute-óptimo, N y D deben escalar juntos: aproximadamente ~20 tokens por parámetro.

Gopher	Chinchilla
parámetros (N)	
280B	70B
tokens (D)	
300B	1.4T
tokens / parám	
~1.1	~20
resultado	
—	gana

El vuelco. antes se apostaba a modelos más grandes; Chinchilla mostró que faltaban datos. ~20 tokens por parámetro.

El giro moderno: sobre-entrenar por la inferencia

Qué optimiza Chinchilla. Solo el costo de entrenar. Minimiza la loss para un compute de training dado, y nada más.

Lo que falta. Un modelo que vas a servir a millones de usuarios se paga también —y sobre todo— en inferencia. Ahí conviene un modelo más chico (más barato de correr) aunque haya que entrenarlo de más.

El resultado en la práctica. Los modelos abiertos sobre-entrenan: Llama 3 8B se entrenó con ~15T tokens (~1900 tokens/param), casi 100× la receta Chinchilla. Modelo chico, muchísima data.

Compute-optimal (Chinchilla)

`~20 tokens / parámetro`

minimiza solo el costo de training

Inference-aware (Llama)

`~1900 tokens / parámetro`

modelo chico, sobre-entrenado: caro una vez, barato de servir por siempre

Por qué importa. 'Chinchilla dice 20' no es una ley universal: es el óptimo cuando solo importa el costo de entrenar. Cuando pesa la inferencia, conviene sobre-entrenar un modelo más chico.

Síntesis: la escala es ciencia, no corazonada

Tres cosas que se llevan. (1) La loss es predecible: power laws. (2) Hay un balance óptimo entre modelo y datos: Chinchilla, ~20 tok/param. (3) Las decisiones reales negocian training contra inferencia.

El hilo entre las dos mitades. Misma epistemología. Los datos: qué usar, decidido con ablations. La escala: cuánto, decidido con scaling laws. Las dos, empíricas.

Los datos — qué usar

decidido con ablations



La escala — cuánto

decidido con scaling laws

empírico: medir, no dogmatizar

La idea que une las dos mitades. datos y escala no se dogmatizan: se miden. Preentrenar es ingeniería empírica a gran escala.

PREENTRENAMIENTO

El modelo base

Qué comprás con todo eso — y por qué todavía no es un asistente.

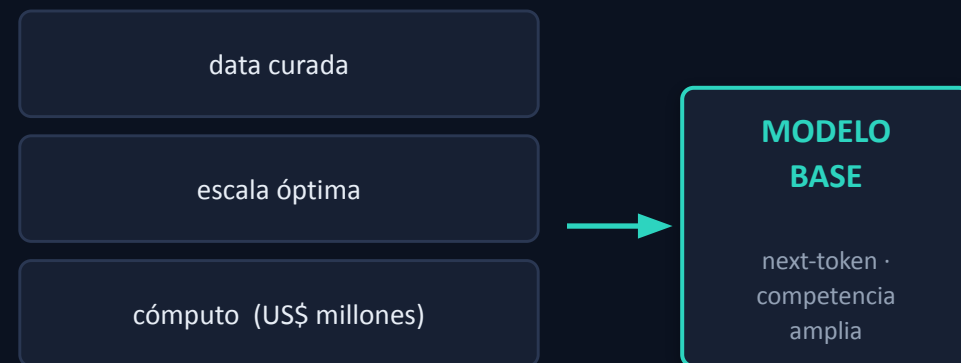


Qué comprás con todo eso: el modelo base

El producto del pretraining. Después de curar la data, elegir la escala y gastar el cómputo, obtenés un modelo base: un predictor de próximo token con competencia amplia.

Una imagen útil. Un 'simulador de documentos de internet'. Aprendió a continuar cualquier texto, y en el camino absorbió una cantidad enorme de conocimiento del mundo.

Lo caro ya está hecho. Toda la competencia del modelo —lo que sabe— se compró acá, de una vez, y a un costo enorme.



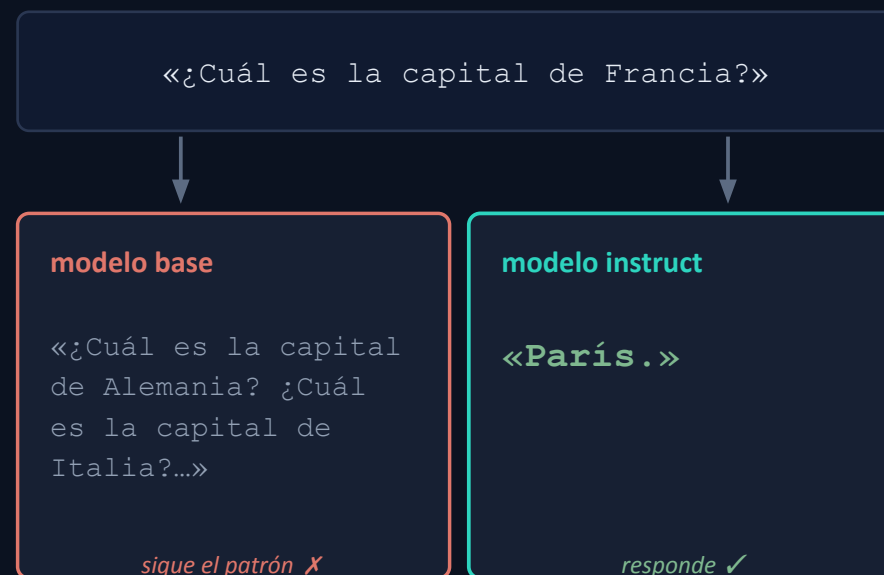
Lo que tenés en la mano. un modelo potentísimo en conocimiento... y todavía crudo en comportamiento.

Lo que NO sabe hacer: comportarse

Su único objetivo fue continuar texto. Nunca se le pidió responder, obedecer ni ser útil: solo predecir el próximo token de documentos. De fábrica, no hace lo que uno espera de un asistente.

El ejemplo que lo muestra. A '¿Cuál es la capital de Francia?', un modelo base puede seguir con '¿Cuál es la capital de Alemania? ¿Cuál es la capital de Italia?'. Completa el patrón —una lista de preguntas— en vez de responder.

No es que no sepa. Sabe perfectamente que es París. El problema no es el conocimiento: es que no fue moldeado para usarlo de forma útil.



El problema a resolver. tiene el conocimiento, le falta el comportamiento. Este ejemplo motiva todo lo que sigue: el fine-tuning.

Por qué pasa, y por qué se arregla barato

El conocimiento ya está adentro. Con un par de ejemplos en el prompt (few-shot), el modelo base responde bien: si le mostrás el formato pregunta→respuesta, contesta 'París'. La capacidad estaba; solo había que activarla.

Dos cosas distintas. El pretraining compró la competencia (lo que sabe): caro, una sola vez. El comportamiento (cómo se presenta) es una capa final mucho más barata.

La proporción. Pretraining: meses y millones. Ajustar el comportamiento: horas o días, con datos y cómputo chicos en comparación.

```
Francia → París  
Alemania → Berlín  
Italia →  
Roma
```

few-shot: el conocimiento ya estaba, solo se activa

COMPETENCIA

lo que sabe
cara · una vez

+

comportamiento

capa final
barata

La buena noticia. lo caro ya está pago. Falta una capa final barata que molde el comportamiento.

Pretraining es un verbo: de cero, o continuado

Nuestro camino fue de cero → fine-tuning. Pero 'pretraining' también nombra algo que le podés hacer a un modelo ya entrenado: seguir pre-entrenándolo con data nueva. Se llama continued pretraining.

Y es la vía que sí está al alcance. Entrenar de cero está fuera del alcance de casi todos (millones de dólares, semanas, billones de tokens). Continuar el pretraining de un modelo abierto sí es hacible con herramientas free/open.

Pero no es de juguete. Upstage construyó Solar, un modelo comercial de ~10B parámetros, exactamente así: partiendo de modelos abiertos y siguiéndolos entrenando.

1 · De cero

pesos random · el preentrenamiento que vimos



2 · Continued pretraining

pesos existentes · mismo objetivo



3 · Fine-tuning

pesos existentes · supervisado

1 y 2 comparten objetivo (next-token); solo cambia de dónde arrancan los pesos.

La aclaración. 'pretraining' no es solo el evento único del preentrenamiento: es un régimen que también se aplica sobre un modelo ya entrenado.

El mecanismo: es el mismo loop, desde un checkpoint

Mecánicamente, no hay nada nuevo. Mismo objetivo next-token, el mismo loop que ya vimos en la parte de transformers (forward → loss → backward → update), sobre texto plano no estructurado, moviendo todos los pesos.

La única diferencia con el preentrenamiento de cero. En vez de inicializar los pesos al azar, arrancás desde un modelo ya entrenado: un checkpoint. Por eso 'continued': literalmente lo continuás.

LO QUE QUEDA IGUAL

- El objetivo: next-token.
- Los datos: texto plano, no estructurado.
- El loop del TP1: forward → loss → backward → update.
- Se mueven todos los pesos del modelo.

LO ÚNICO QUE CAMBIA

inicialización: pesos random → **checkpoint**

La clave. no es una técnica nueva: es el mismo pretraining, resumido desde un checkpoint. Lo único distinto es de dónde arrancan los pesos.

Continued pretraining vs fine-tuning: no confundir

La distinción que importa. Continued pretraining agrega competencia / conocimiento (lo que el modelo sabe). Fine-tuning agrega comportamiento (cómo se presenta) — el mismo par de las láminas anteriores.

El caso que lo hace tangible. 'Querés que hable coreano' es conocimiento nuevo: no lo conseguís instruyéndolo con ejemplos, necesitás seguir pre-entrenándolo sobre corpus coreano.

Por qué no se sustituyen. Un dataset chico de instrucciones no alcanza para meter un dominio o un idioma entero. Cada herramienta compra una cosa distinta.

Continued PT	Fine-tuning
para qué	
conocimiento (dominio/idioma)	comportamiento (responder, formato)
objetivo	
next-token (self-sup.)	pares supervisados
datos	
texto plano, mucho	pares, poco
costo	
US\$ cientos de miles	chico

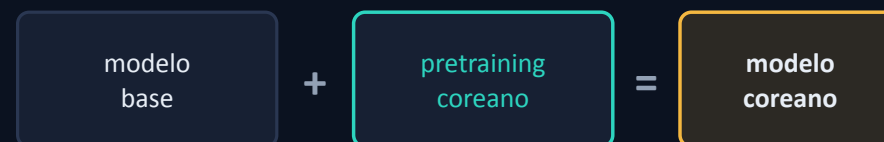
La regla. conocimiento nuevo → seguí pre-entrenando; comportamiento nuevo → fine-tuning. No metés un dominio entero con un dataset chico de instrucciones.

En la práctica: modelo + pretraining coreano = modelo coreano

El ejemplo que lo dice todo. Modelo base + continued pretraining sobre corpus coreano = un modelo que sabe coreano. Un idioma nuevo es conocimiento nuevo en estado puro: el 'cuándo' más claro para seguir pre-entrenando.

Los números (Solar coreano, Upstage). Continued pretraining sobre ~200B tokens, ~US\$0.2M. De cero, ese modelo habría necesitado billones de tokens y millones de dólares: mucho más barato, mismo tamaño.

El cuidado. No es 'puro coreano': mezclan coreano + inglés, justo para que el modelo no se olvide del inglés que ya sabía (catastrophic forgetting).



costo / data (crece con lo que pedís):

fine-tuning: data chica



continued PT: ~200B tok · ~US\$0.2M



de cero: ~billones tok · US\$ millones



Ejemplo abierto y reproducible: OPEN-SOLAR-KO (HuggingFace). Números: curso Pretraining LLMs (DeepLearning.AI). Método: paper SOLAR, arXiv 2312.15166.

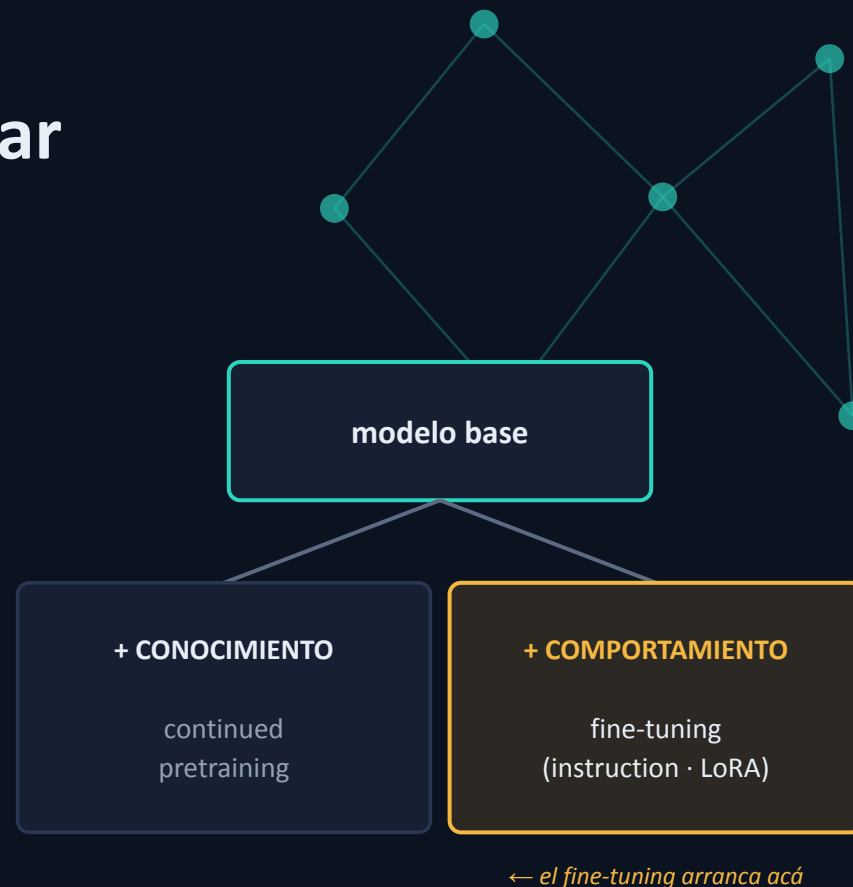
Lo que se llevan. es la vía realista para especializar por conocimiento — cuidando no pisar lo viejo.

El puente al fine-tuning: dos ejes para adaptar

Adaptar un modelo base se parte en dos. Agregar conocimiento (continued pretraining, lo que acabamos de ver) o agregar comportamiento (fine-tuning). Son ejes distintos, no rivales: cada uno arregla una cosa.

Qué sigue. El fine-tuning se enfoca en el eje del comportamiento: qué comportamiento darle → instruction tuning; cómo hacerlo barato, sin re-tocar todo el modelo → PEFT, LoRA, QLoRA.

Dónde estamos. Hoy cerramos el preentrenamiento: data + escala → modelo base. El fine-tuning lo convierte en un asistente.



El cierre. el pretraining entrega competencia cruda; para especializarla hay dos caminos — conocimiento (más pretraining) o comportamiento (fine-tuning). El fine-tuning toma el segundo.